三系统统一用户管理方案（讨论稿）

日期：2026-09-14 范围：台账（taizhang，已生产/Oracle）、固定资产（gudingzichan01，未部署）、数字画像（digital_portrait，未部署）

一、现状摸底结论

	台账	固定资产	数字画像
技术栈	FastAPI 同步 + React/antd	FastAPI 同步 + React/antd	FastAPI **异步** + React/antd
业务库	Oracle（可切 SQLite）	SQLite	SQLite（字段兼容 SQL Server）
登录标识	7 位 EHR 号	7 位 EHR 号	7 位 EHR 号
密码哈希	passlib bcrypt	bcrypt	PBKDF2-SHA256（10 万次 +salt，最规范）
Token 有效期	8 天	30 天	1 小时
角色模型	is_superuser + Casbin（半成品，端点多未启用）	role 字段：admin/leader/user + 组	role 字段：admin/leader/user + 组
密码策略	3 个月过期	首登强改	强度校验 + 首登强改 + 默认密码

关键事实：

- 三个系统用户为**同一批人**，均以 EHR 号为唯一标识 → 一张用户表可行，无需按系统做用户子集。
- 固定资产与数字画像**尚未部署**，auth 模块改造成本处于历史最低点。
- 台账 Casbin RBAC 是半成品（policy 只有一行、多数端点权限检查被注释），无论选哪个方案都建议**废弃重构**，不要在它上面续建。
- 两个待部署系统有 SECRET_KEY 弱默认值、token 过长等问题，统一改造时一并修复。

二、已确认的需求共识

1. 统一程度：**L1（账号密码一处管）+ L3（角色一处配）**，暂不做 L2 单点登录跳转。
2. 角色模型：**用户 × 系统 × 角色** 三元组（路线 B），各系统角色语义不强求对齐；保留极少数**全局超管**（离职停用等全局操作）。
3. 密码策略以数字画像标准为基线（强度校验 + 首登强改 + 默认密码），**保留台账的 3 个月密码过期**（全局生效）。Token 有效期统一 8 小时。
4. 固定资产与数字画像**不再提供**新建用户 / 改密码入口，用户管理操作全部收进中心。
5. 存量密码不迁移：首次切换时全员重置为默认密码 + 首登强制改密。
6. 运维约束：内网机器紧张，**倾向少一个服务进程** → 台账为中心是主方案；独立认证服务作为分支保留。

三、方案

方案一（主推）：台账兼任用户中心，API 同步、密码不落地

台账后端新增一组内部认证 API（挂现有 8080 服务，零新增进程）：

```
POST /internal/auth/verify      {ehr, password, system} → {ok,
user_profile, roles_in[system], password_expired, must_change}
GET  /internal/users/snapshot   → 全量用户+角色（初始导入与对账用，可带版本号增
量）
POST /internal/users/push        （可选）变更推送，如停用即时生效
```

- 固定资产 / 数字画像：
  - 删除本地密码字段，登录改为调用 verify 接口；
  - 本地保留用户表（只存资料 + 角色缓存 + 状态），verify 成功后用返回值刷新本地行——Oracle 短暂不可用时，靠本地缓存的角色照常放行已登录用户（容错可做成一个 try/except 降级，代码量极小）；
  - 角色判断逻辑（leader 的组范围等）不动，只把角色来源从本地字段换成中心返回。
- 台账自身：新建 `system_role(user_id, system, role)` 表承接路线 B；废弃 Casbin，权限检查改为读该表 + 保留 is_superuser 作全局超管。
- 改造量估计（两系统均未部署，是最佳时机）：
  - 台账：auth 模块重构 + 用户管理界面扩展成三系统赋权矩阵 + Casbin 拆除，约 3~5 人日；
  - 固定资产 / 数字画像：各改 auth 模块（登录转发 + 本地表瘦身 + 去掉用户管理入口），各约 2~3 人日。

优点：

- **零新增进程**，符合机器紧张约束；复用台账已上线的 Oracle 和运维。
- 不存在密码同步环节（同步哈希/明文的安全坑全部消失），三种哈希算法的历史包袱自然作废。
- 两系统**无需迁移 Oracle**（见第五节），业务库留 SQLite，不用装 Oracle 客户端驱动，不用处理异步驱动的兼容问题。
- Oracle 故障只影响"新登录"，不影响已登录用户。

缺点 / 风险：

- 台账成为关键依赖：台账后端停机维护期间，另两系统无法新登录（已登录不受影响）。内网系统可接受，需告知运维。
- 业务系统与基础设施耦合在台账里，未来若台账大改版，用户中心要跟着动（缓解：把 internal API 独立成台账内的一个 router 模块，边界清晰，将来可整体平移出去）。
- 台账 auth 模块本身要动手术（Casbin 拆除），动的是生产系统，需要回归测试。

方案二（分支）：独立轻量统一认证服务

单独一个 FastAPI 服务 + 自己的库（Oracle 或 SQLite 均可），对外提供与方案一完全相同的 internal API；台账降级为普通接入方。

- 优点：架构最干净，认证与业务解耦；台账不用动 auth（Casbin 可以不拆，只把用户来源切到中心）；第四个系统接入时零心理负担；未来若做 L2 单点登录，演进路径最顺。

- 缺点：多一个常驻进程和部署单元（与"机器紧张"冲突）；用户管理界面要在新服务里从零做一遍（账号、赋权矩阵、审计日志）；台账已有的用户管理代码大部分作废重写；总改造量比方案一多约 30~50%。
- 适用触发条件：如果讨论中有人强烈认为"生产台账不该承担基础设施职责"，或已有第四系统的明确规划，选此方案。

> 两方案的 internal API 契约设计成完全一致——这是刻意的：先按方案一落地，将来要升级到方案二，只需把这套 router 从台账里平移到独立服务，另两系统只改一个 base_url。

方案三（参考）：台账做成 ERP，把另外两系统并进来

代码与数据库层面的改造量评估：

- 代码层：
  - 三者后端同为 FastAPI，但台账/固定资产是同步 SQLAlchemy，数字画像是全异步（AsyncSession + aiosqlite）——**异步与同步代码无法直接合并**，要么把数字画像全量改同步（改写所有 router/service，约等于重写后端的一半），要么台账整体转异步（动生产系统，风险不可接受），要么同一进程内双引擎并存（反模式，维护成本更高）。
  - 三套 SQLAlchemy models 存在表名/审计字段/软删除约定冲突，需要逐表核对合并。
  - 前端三者同为 React+antd+Vite，合并为一个多页应用技术上顺滑（router 分模块即可），这是三者中最容易的部分。
- 数据库层：业务数据要么全部迁 Oracle（固定资产/数字画像的迁移工具都要新写），要么单库多数据源（事务边界、跨库关联都要绕）。
- 总量级估计：**20~40 人日起步，且数字画像部分接近重写**；而方案一 + 保持三系统分离是 8~11 人日。
- 结论：**不建议以统一用户管理为由做 ERP 化**。ERP 化的合理动机应是功能层面的深度联动（如台账数据自动生成资产卡片），若将来真有此需求，也应先按方案一/二统一认证，再逐模块合并——顺序反了会同时锁死三个系统的迭代。

四、角色模型（两方案通用）

- `users`：一张全局用户表（EHR 号唯一键、姓名、组、状态、密码哈希、密码时间）。
- `user_system_role(user_id, system, role)`：路线 B 三元组，system ∈ {taizhang, assets, portrait}，各系统自定义角色集。
- 全局超管：users 上一列（或 role='GLOBAL_ADMIN'），可跨系统操作用户生命周期。
- 赋权界面：人员 × 系统矩阵，每格角色下拉；支持"某人在所有系统同为 admin"一键操作。
- 离职/停用：中心停一处，三个系统同时失效（停用后 verify 直接拒绝；配合可选的 push 让已发 token 提前失效——JWT 无法吊销，若要求秒级生效需在请求校验里查缓存状态，建议先接受"最长 8 小时残留"，与 token 统一有效期配套）。

五、"另外两系统迁 Oracle"的必要性评估

结论：**方案一下无必要。**

- 密码在台账 Oracle，业务数据（资产卡片、画像档案）量小且归属单一系统，SQLite 完全胜任；
- 只有当出现以下任一情况才值得迁：① 集中备份/容灾的合规硬要求；② 需要跨系统 SQL 关联分析（如"资产台账与画像联合报表"）——即便如此，也应优先考虑数据仓库式的抽取，而不是业务库直连；
- 若选方案二且认证服务想直接用 Oracle，那只涉及认证服务自己，与两业务系统无关。

六、实施顺序建议（以方案一为例）

1. 台账：建 system_role 表 + internal auth router + 赋权矩阵界面 + 拆 Casbin + 全员重置默认密码（上线一个版本，自身先跑通）。
2. 数字画像 / 固定资产（开发期直接改）：删本地密码与用户管理入口、登录接 verify、角色读中心缓存 —— 各自随首次部署上线，无存量迁移。
3. 统一 SECRET_KEY 为强随机值并持久化、token 有效期统一 8 小时（顺带修掉两处弱默认 SECRET）。
4. 总投入约 8~11 人日；此后入职/离职/赋权/改密均为一处操作。